

# C++ 核心知识点易自学手册（第 4 章完整版）

## 前言

本文档基于原书第 4 章“指针、数组、结构与动态内存”整理，**全面覆盖所有核心知识点**，采用“概念 + 代码 + 对比 + 避坑”的四维度解析模式，适合打印后逐节学习。**每节包含自测练习**，确保知识点掌握。

## 4.7 指针和自由存储空间

### 1. 指针的本质与核心符号

- 指针定义**：存储内存地址的变量，通过地址间接访问数据。
  - 类比：身份证号（地址）→ 对应个人信息（值）。
- 核心符号**：

| 符号 | 名称   | 作用        | 示例                                      |
|----|------|-----------|-----------------------------------------|
| &  | 取地址符 | 获取变量的内存地址 | <code>int a=10; int* p = &amp;a;</code> |
| *  | 解引用符 | 获取指针指向的值  | <code>cout &lt;&lt; *p; // 输出10</code>  |

- 内存可视化**：

|        |                     |
|--------|---------------------|
| 内存地址   | 存储内容                |
| 0x1000 | 10 ← a 的值           |
| 0x2000 | 0x1000 ← p 的值（即 &a） |

### 2. 指针的声明与初始化

- 声明格式**：类型\* 指针名；（类型决定指针操作的数据大小）。
- 初始化方式**：

| 方式      | 代码示例                                    | 说明        |
|---------|-----------------------------------------|-----------|
| 指向已有变量  | <code>int a=10; int* p = &amp;a;</code> | 最安全的初始化方式 |
| 动态分配内存  | <code>int* p = new int;</code>          | 需手动释放内存   |
| 初始化为空指针 | <code>int* p = nullptr;</code>          | 避免野指针     |

◦ **错误案例：**

```
int* p;    // 未初始化的指针（野指针）
*p = 100;  // 危险！可能导致程序崩溃
```

3. 动态内存：new 与 delete

◦ **new 操作符：**

| 场景     | 语法                                | 示例                                  |
|--------|-----------------------------------|-------------------------------------|
| 分配单个对象 | <code>类型* 指针 = new 类型;</code>     | <code>int* p = new int(5);</code>   |
| 分配数组   | <code>类型* 指针 = new 类型[长度];</code> | <code>int* arr = new int[3];</code> |

◦ **delete 操作符：**

| 场景     | 语法                        | 示例                         |
|--------|---------------------------|----------------------------|
| 释放单个对象 | <code>delete 指针;</code>   | <code>delete p;</code>     |
| 释放数组   | <code>delete[] 指针;</code> | <code>delete[] arr;</code> |

◦ **内存泄漏示例：**

```
int* p = new int;
// 忘记 delete p; → 内存泄漏（程序结束前无法回收）
```

4. 动态数组 vs 静态数组

| 特性 | 静态数组 | 动态数组 |
|----|------|------|
|----|------|------|

长度确定时间

| 编译时（固定） |                          | 运行时（可变参数）                           |
|---------|--------------------------|-------------------------------------|
| 内存位置    | 栈区                       | 堆区                                  |
| 长度是否可变  | 否                        | 否（但可重新分配新数组）                        |
| 声明方式    | <code>int arr[5];</code> | <code>int* arr = new int[5];</code> |

## 4.8 指针、数组和指针算术

### 1. 指针与数组的等价性

- 核心结论：

| 表达式                      | 含义                              |
|--------------------------|---------------------------------|
| <code>arr</code>         | 数组首元素地址（类型为 <code>int*</code> ） |
| <code>arr[i]</code>      | 等价于 <code>*(arr + i)</code>     |
| <code>&amp;arr[i]</code> | 等价于 <code>arr + i</code>        |

- 代码验证：

```
int arr[3] = {1, 2, 3};
int* p = arr; // p指向arr[0]
cout << arr[1] << endl; // 输出2
cout << *(arr + 1) << endl; // 输出2（等价于arr[1]）
cout << p[1] << endl; // 输出2（指针可用数组下标）
```

### 2. 指针算术的特殊性

- 规则：指针  $\pm 1$  时，移动的字节数 = 指向类型的大小（如 `int` 占 4 字节，`double` 占 8 字节）。

| 指针类型              | <code>p + 1</code> 移动的字节数 | 示例地址变化（假设初始地址 0x1000） |
|-------------------|---------------------------|-----------------------|
| <code>int*</code> | 4                         | 0x1000 → 0x1004       |
|                   |                           |                       |

|                      |   |                 |
|----------------------|---|-----------------|
| <code>double*</code> | 8 | 0x1000 → 0x1008 |
| <code>char*</code>   | 1 | 0x1000 → 0x1001 |

### 3. 字符串与指针

- **C 风格字符串**：以 `\0` 结尾的字符数组（`\0` 是结束标志，不计数）。
- **字符串操作对比**：

| 操作   | 普通数组方式                             | 指针方式                                  |
|------|------------------------------------|---------------------------------------|
| 定义   | <code>char str[] = "hello";</code> | <code>const char* p = "hello";</code> |
| 长度计算 | <code>strlen(str)</code>           | <code>strlen(p)</code>                |
| 输出   | <code>cout &lt;&lt; str;</code>    | <code>cout &lt;&lt; p;</code>         |

- **易错点**：

```
char str[5] = "hello"; // 错误！"hello"含5个字符+1个\0，需6字节
cout << str; // 可能输出乱码（缺少\0结束符）
```

### 4. 动态结构

- **结构指针的访问方式**：

| 方式        | 语法                     | 示例                           |
|-----------|------------------------|------------------------------|
| 解引用后访问    | <code>(*指针).成员</code>  | <code>(*p).age = 20;</code>  |
| 箭头操作符（推荐） | <code>指针-&gt;成员</code> | <code>p-&gt;age = 20;</code> |

- **示例代码**：

```
struct Point {
    int x, y;
};
Point* p = new Point; // 动态创建结构
p->x = 10;
p->y = 20;
```

```
cout << p->x << ", " << p->y << endl; // 输出: 10, 20
delete p; // 释放结构
```

## 4.9 类型组合（结构、数组、指针的混合使用）

### 1. 结构数组

- 定义与访问：

```
struct Student {
    char name[20];
    int score;
};
// 初始化结构数组
Student students[3] = {
    {"Alice", 90},
    {"Bob", 85},
    {"Charlie", 95}
};
cout << students[1].name << endl; // 输出: Bob
```

### 2. 指针数组（数组元素为指针）

- 用途：存储多个字符串（避免二维数组的固定长度限制）。

```
const char* names[3] = { // 每个元素是字符串的地址
    "Tom",
    "Jerry",
    "Spike"
};
cout << names[1] << endl; // 输出: Jerry
```

### 3. 指向指针的指针（二级指针）

- 用途：处理指针数组（如遍历、修改指针指向）。

```
int a = 10;
int* p = &a;    // 一级指针：指向int
int** pp = &p;  // 二级指针：指向int*
cout << **pp << endl; // 输出：10（等价于*p → a）
```

## 4.10 数组的替代品（vector 与 array）

### 1. 模板类 vector（动态数组）

- 核心特性：

| 特性   | 说明                                                                                       |
|------|------------------------------------------------------------------------------------------|
| 动态扩容 | 长度按需增长（通常按原容量的 2 倍扩容）                                                                    |
| 内存位置 | 堆区                                                                                       |
| 常用方法 | <code>push_back()</code> （末尾添加）、 <code>size()</code> （元素数）、 <code>at()</code> （带越界检查的访问） |

- 示例代码：

```
#include <vector>
using namespace std;
vector<int> v; // 空vector
v.push_back(10); // 添加元素
v.push_back(20);
cout << v.size() << endl; // 输出：2（当前元素数）
```

### 2. 模板类 array（固定长度数组）

- 核心特性：

| 特性   | 说明                                              |
|------|-------------------------------------------------|
| 固定长度 | 编译时通过模板参数指定（如 <code>array&lt;int,3&gt;</code> ） |
|      |                                                 |

|      |                  |
|------|------------------|
| 内存位置 | 栈区（效率同普通数组）      |
| 安全性  | 提供at()方法，越界时抛出异常 |

◦ 示例代码：

```
#include <array>
using namespace std;
array<int, 3> arr = {1, 2, 3}; // 长度固定为3
cout << arr.at(1) << endl; // 输出：2（带越界检查）
```

3. 三种数组对比表

| 特性    | 普通数组       | vector          | array      |
|-------|------------|-----------------|------------|
| 长度灵活性 | 固定（编译时）    | 动态（运行时）         | 固定（编译时）    |
| 内存位置  | 栈区         | 堆区              | 栈区         |
| 越界检查  | 无（直接访问）    | 有（at()方法）       | 有（at()方法）  |
| 初始化方式 | 手动赋值或列表初始化 | 动态添加（push_back） | 列表初始化      |
| 适用场景  | 长度固定且简单场景  | 长度不确定的动态数据      | 长度固定但需安全检查 |

4.11 内存管理方式总结

1. 三种内存区域对比

| 区域       | 分配方式        | 释放方式       | 生命周期          | 示例                |
|----------|-------------|------------|---------------|-------------------|
| 栈（stack） | 自动分配（函数内变量） | 函数结束时自动释放  | 函数执行期间        | int a;            |
| 堆（heap）  | new手动分配     | delete手动释放 | 程序员控制（不释放则泄漏） | int* p = new int; |

静态区

|                   |               |              |               |
|-------------------|---------------|--------------|---------------|
| 全局变量<br>或static修饰 | 程序结束时自动<br>释放 | 整个程序运行期<br>间 | static int a; |
|-------------------|---------------|--------------|---------------|

## 2. 内存泄漏的危害与避免

- **危害：**程序占用内存越来越多，严重时导致崩溃。
- **避免方法：**
  - new与delete、new[]与delete[]严格配对。
  - 使用智能指针（如unique\_ptr，自动管理内存）。

## 重点警告与自查清单

- 1. 指针初始化：**永远不要使用未初始化的指针（野指针）。
- 2. 内存配对：**new与delete、new[]与delete[]必须配对，禁止重复释放。
- 3. 数组越界：**C++ 不检查数组越界，访问前务必确认下标范围。
- 4. 字符串结束符：**C 风格字符串必须以\0结尾，避免cout或strlen读取异常。
- 5. 指针数组操作：**遍历指针数组时，注意每个指针是否指向有效地址。

## 自测练习（覆盖所有核心点）

1. 声明一个指向double的指针，用new分配内存并赋值 3.14，最后释放。
2. 解释arr[3]与\*(arr+3)为何等价，并编写代码验证。
3. 用动态数组实现：输入 n 个整数，输出最大值（n 由用户输入）。
4. 定义一个包含name（字符串）和score（double）的Student结构，用动态内存创建 1 个学生并赋值，最后释放。
5. 对比vector和普通数组的区别，各举一个适用场景。
6. 说明char str[] = "hello";中省略数组长度的原理，若写成char str[5] = "hello";会有什么问题？
7. 编写代码，使用二级指针修改一个整数的值。
8. 找出以下代码的内存泄漏点并修复：



```
void func() {  
    int* p = new int[10];  
    p = new int[20]; // 问题所在  
}
```

9. 编写代码验证vector的扩容机制（提示：使用capacity()方法）。
10. 使用array重写一个普通数组的冒泡排序，体验at()方法的越界检查功能。  
（注：文档部分内容可能由 AI 生成）